Detecting Weight Tampering in Verifiable LLM Inference: Per-Layer Activation Profiles and the Attention-Kernel Loophole

Vignesh Kanike
Indian Institute of Technology Delhi
ms1240669@iitd.ac.in

Abstract

Clients of inference providers usually cannot tell whether the open-weight model they paid for was run, or a cheaper quantised or fine-tuned copy. TOPLOC (Ong et al., ICML 2025) lets a provider commit to the largest entries of the last hidden state, which a validator checks with one prefill, using thresholds loose enough to absorb honest numerical noise. On Qwen2.5-1.5B-Instruct we find that this tolerance is the weak point: serving the reference model with eager attention changes the last layer more than 8-bit or 4-bit quantisation, so thresholds that accept it flag no quantised sequence and 55–60% of fine-tuned ones. Kernel noise and weight changes differ across depth, and a linear classifier on TOPLOC's statistics at all 28 layers attributes four known modifications with 99.3% balanced accuracy (99.0% from decode-time activations), against 78.0% from the last layer alone. We then test it as an integrity check against three kinds of provider. It is not one in general: 8-bit quantisation left out of training is accepted on every test prompt, and a provider that serves quantised weights with eager attention is accepted on 92% (8-bit) and 68% (4-bit) of prompts at a 1% false-rejection target. With the kernel enforced to SDPA, no tampered model served on it was accepted: not the four known modifications at batch 1 or 4, six unseen partial or combined modifications, or a detector-guided choice of layers to quantise (0 of 50 per case). On a second model, SmolLM2-1.7B, eager noise is small, the last layer alone suffices, and the loophole does not appear. A per-layer TOPLOC commitment costs 57.8 kB per 256 tokens. Neither check binds the served text to the weights that generated it, so the results bound what activation checks can establish rather than provide an integrity guarantee.

1 Introduction

Open-weight language models are now served by many third parties: commercial APIs, decentralised compute networks and peer-to-peer inference markets. A client names a model and pays per token, but it sees only text. A provider that quantises the model or swaps in a fine-tuned copy saves memory or compute, and the text still reads well. Gao et al. [6] tested 31 commercial endpoints for four Llama models and found 11 serving a different output distribution from the published weights.

TOPLOC [10] offers a cheap check. Every 32 tokens the provider commits to the 128 largest entries of the last hidden state, and a validator recomputes them in a single prefill. Because bf16 arithmetic differs slightly between GPUs and kernels, the validator accepts small exponent and mantissa disagreements and flags a sequence only above fixed thresholds. That tolerance is where the difficulty lies. A verifier has to accept every authorised execution, including honest providers on different kernels, and reject every unauthorised one. If honest variation and weight tampering produce disagreements of the same size, no threshold does both.

We find exactly this overlap on Qwen2.5-1.5B-Instruct. Serving the reference model with eager attention, a kernel TOPLOC did not evaluate, changes the last layer about six times as much as a batch-size change and more than 8-bit or 4-bit quantisation. With thresholds calibrated to accept eager traffic, the check flags no quantised sequence and 55–60% of fine-tuned ones.

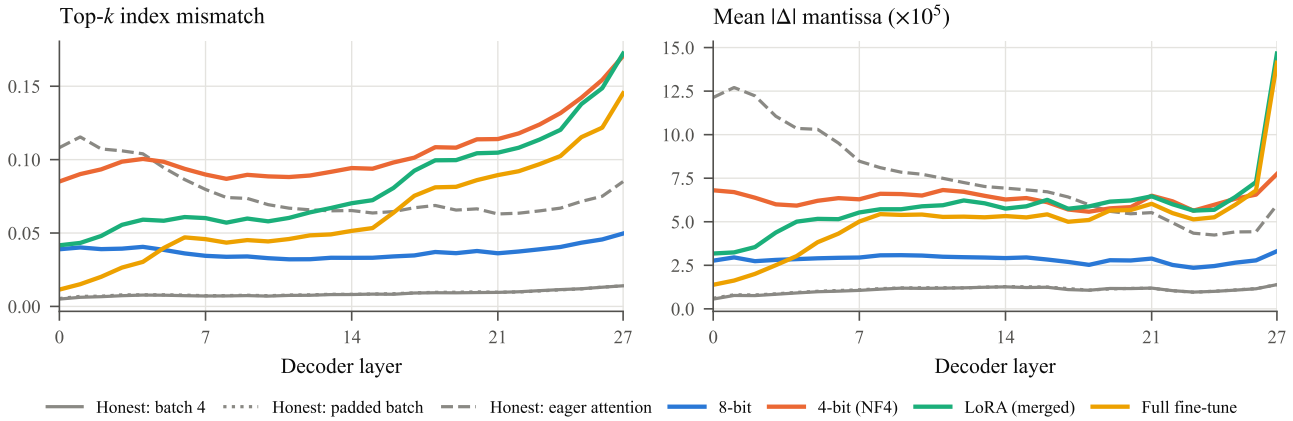


Figure 1: Kernel noise and weight changes peak at different depths. Mean top-k disagreement with the reference model at each decoder layer (Qwen2.5-1.5B-Instruct, 200 prompts). Grey lines are honest serving configurations; coloured lines are tampered weights. At layer 27, eager-attention noise sits among the quantised models; in the early layers it stands apart.

The two sources do differ in where they appear (Figure 1). Eager-attention noise is largest in the first layers and fades with depth, quantisation shifts every layer by about the same amount, and fine-tuning grows toward the output. We compute TOPLOC's statistics at all 28 layers and train a linear classifier on the resulting profile. On the four modifications we trained on, it identifies the modification with 99.3% balanced accuracy while accepting both kernels.

That number answers a narrower question than integrity verification, and most of this paper is about the gap. We evaluate the profile against three kinds of provider (Section 3): one that tampers without regard to the detector, one that also chooses its kernel and batch size, and one that uses the detector's own score to choose what to quantise. We also test modifications the classifier never saw, report false acceptance and false rejection rates at a fixed operating point with intervals, and separate what our implementation checks from what a deployed protocol would still have to guarantee.

Our findings, each tested in Section 6:

- With thresholds calibrated on mixed honest kernels, TOPLOC's last-layer check misses 8-bit and 4-bit quantisation and about half of light fine-tuning in our setup (Section 6.1).
- The per-layer profile identifies four *known* modifications with 99.3% balanced accuracy, 99.0% from decode-time activations, and most of its signal lies in the early and middle layers (Sections 6.2 to 6.4).
- It does not establish integrity against unknown modifications: 8-bit quantisation left out of training is accepted as clean on every test prompt (Section 6.6).
- A provider that serves quantised weights with eager attention passes as honest on most prompts, even when the verifier uses a detector specific to the declared kernel. With the kernel enforced, no tested attack was accepted (Section 6.7).
- Six modifications never seen in training, including LoRA with 4-bit weights and partial quantisation, and a detector-guided choice of layers to quantise were all rejected with the kernel enforced; on SmolLM2-1.7B the kernel loophole does not appear and the last layer alone suffices, so both the problem and the benefit are model-dependent (Sections 6.6, 6.8 and 6.9).
- A real per-layer TOPLOC commitment costs 57.8 kB per 256 tokens and 0.5 s of CPU time per sequence, and at batch 32 4-bit decoding reaches 96% of bf16 throughput with less memory (Section 6.10).

Code and result files are at `https://github.com/bruce249/proof-of-weights`.

2 Background and related work

TOPLOC. TOPLOC [10] records the last hidden state at one position per 32 decode steps, keeps its k=128 largest-magnitude entries, and encodes them in a polynomial commitment of a few hundred bytes. The validator prefills the prompt and the claimed output with the committed weights, counts exponent mismatches between corresponding values, and takes the mean and median mantissa difference. A sequence fails if the largest value of any statistic across its checkpoints exceeds a threshold; the published thresholds are T_{exp}=38, T_{mean}=10 and T_{median}=8. Its robustness tests covered A100 and RTX 4090 GPUs, Flash Attention 2, SDPA and FlexAttention, and tensor parallelism on Llama-3.1-8B and larger models, with no false positives [10, §5.6, Table 2]. It did not test quantisation, notes that small fine-tuning updates are harder to detect [10, §6.5], and cannot detect speculative decoding [10, §6.2].

Statistical detection versus proof. Zero-knowledge proofs of inference such as zkLLM [12] establish that a specific computation was performed, at minutes per query for a 13B model; SafetyNets [7] uses interactive proofs for smaller networks. Trusted execution [1, 16] instead relies on the hardware vendor. TOPLOC and the method in this paper are statistical tests: they bound how far observed activations may deviate from a recomputation, and their guarantees are only as strong as the separation between honest and dishonest deviations, which is what we measure.

Detecting substitution. Model equality testing [6] compares output text with a two-sample test; Cai et al. [1] show such tests need many queries and that log-probability tests are confused by nondeterministic inference. SVIP [13] trains a proxy task on returned hidden states to detect substitution with a smaller model. These methods target swaps between different models, whereas we study quantisation and fine-tuning of the same model.

Nondeterminism and enforcement. Inference is not bitwise reproducible across batch sizes, padding and kernels [8], so every activation check needs some tolerance. Optimistic protocols such as Truebit [15] and opML [2] accept results by default and settle disputes on challenge; TOPLOC has no stake or penalty.

3 Threat model and problem

Assets and parties. The asset is the client's assurance that a response was produced by the committed weights θ, a model repository at a pinned revision. A *provider* stakes tokens, serves requests and posts a commitment for each response. A *validator* samples commitments, recomputes activations with θ, and resolves challenges. We trust the validator to run θ and to report its verdict honestly. The contract enforces the stake, the bond and the dispute window; it does not check the verdict.

Authorised variation. An honest provider runs θ but may use a different attention kernel, batch size or padding from the validator. In our experiments the authorised set is SDPA at batch 1 (the validator's own setting), eager attention at batch 1, SDPA at batch 4, and SDPA at batch 4 with extra padding.

Attacker. A dishonest provider wants to serve weights $\tilde{\theta} \neq \theta$ to save memory or compute, or to change behaviour, without losing its stake. It controls its weights, kernel and batch configuration, and computes its commitments honestly from what it actually ran. We distinguish three levels of knowledge and control:

- **A1, non-adaptive**: serves a quantised or fine-tuned model on the reference kernel, without regard to the detector.
- **A2, configuration-adaptive**: additionally chooses an authorised kernel or batch size to look honest.
- **A3, detector-adaptive**: knows the detector and uses its score to choose the modification.

We evaluate A1 and A2 fully. For A3 we evaluate one constrained strategy, choosing which layers to quantise (Section 6.8); we do not optimise weights against the detector by gradient, and we make no claim of robustness to that.

Out of scope. We do not address forged or replayed commitments, a provider that answers verification samples differently from ordinary traffic, or a provider that generates with $\tilde{\theta}$ and then computes its commitment by a prefill with θ. The last case defeats both TOPLOC and our check, and is discussed in Section 7.

Detection and attribution. Integrity verification is a binary decision: accept a sequence as produced by θ under authorised variation, or reject it. We report it as the false acceptance rate (FAR, tampered sequences accepted) and the false rejection rate (FRR, honest sequences rejected). Attribution, naming which known modification was made, is a separate closed-set task and is reported as balanced accuracy.

4 Method

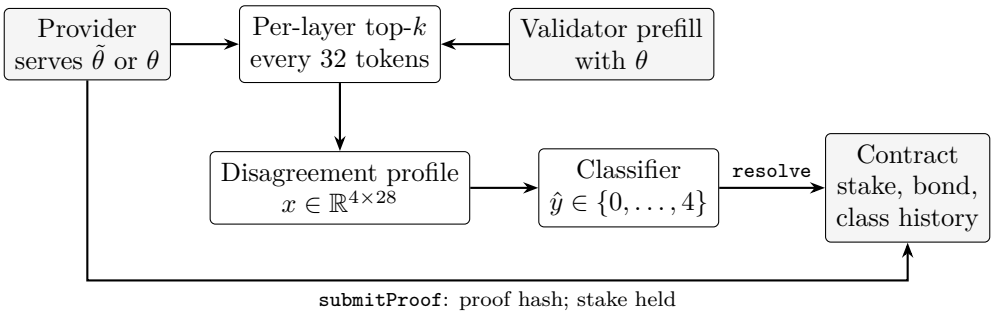


Figure 2: Overview. The provider's per-layer top-k activations are compared with a validator prefill under the committed weights (Section 4.1). A classifier turns the profile into a verdict (Section 4.2), and an upheld challenge slashes the stake and stores the label (Section 4.3).

4.1 Per-layer disagreement profile

Let $h_t^{(\ell)} \in \mathbb{R}^{1536}$ be the output of decoder layer $\ell \in \{0, \ldots, 27\}$ at position t. At each checkpoint $t_c = |\text{prompt}| + 32c$ with $c = 0, \ldots, 7$, we take the indices I and values v of the 128 largest-magnitude entries of $h_{t_c}^{(\ell)}$, for the model under test and for θ on the same tokens. For each checkpoint and layer we compute the overlap of the index sets and TOPLOC's three statistics:

$$
\begin{aligned}
f_{\text{idx}} &= 1 - \tfrac{1}{k}\,|\tilde{I} \cap I|, \\
f_{\text{exp}} &= \textstyle\sum_i \mathbb{1}[\exp(\tilde{v}_i) \neq \exp(v_i)], \\
f_{\text{mean}} &= \text{mean}_i\,|\,\text{mant}(\tilde{v}_i) - \text{mant}(v_i)|, \\
f_{\text{med}} &= \text{median}_i\,|\,\text{mant}(\tilde{v}_i) - \text{mant}(v_i)|.
\end{aligned}
$$

Here exp and mant are the exponent and mantissa fields of the float32 value, and i runs over rank. Averaging over the eight checkpoints gives a profile $x \in \mathbb{R}^{4\times 28}$. Its last row is what TOPLOC already uses.

4.2 Classifier and decision rule

We train a multinomial logistic regression with balanced class weights on profiles labelled clean or with one of four modifications, with features standardised on the training data. A linear model is easy to inspect and needs only a few hundred examples per class. The predicted class gives attribution. For the binary verdict we accept a sequence when $P(\text{clean}) \geq \tau$. The threshold τ is chosen on training prompts only: we compute out-of-fold $P(\text{clean})$ for honest training sequences by prompt-grouped cross-validation and set τ at their 1st percentile, a target FRR of 1%. Test prompts are never used to choose τ.

4.3 Verification protocol

Table 1 separates what we implemented from what a deployment would need. TOPLOC's last-layer commitment, the validator's recomputation, the classifier and the staking contract are implemented; a per-layer commitment is not integrated into the protocol, and its cost is measured separately (Section 6.10). In our main experiments the provider side is also a prefill, which isolates the effect of the weights; Section 6.4 repeats them with decode-time activations, which is what a provider would commit to.

The contract (142 lines of Solidity) stores each provider's model hash, stake and history of tamper classes, and each proof's prompt hash, proof hash and timestamp. A challenger posts a bond within two hours of a proof, and only the validator can resolve it. If upheld, the stake is split between challenger and validator, the bond is returned and the class is recorded (0 clean, 1 8-bit, 2 4-bit, 3 LoRA, 4 fine-tune, 255 unattributed); otherwise the validator keeps the bond. For a sampling rate p, stake S and a saving Δ per request from tampering, a risk-neutral provider facing certain detection on sampled requests is deterred when $\Delta < pS$. The system is not trustless: its verdicts are as trustworthy as the validator.

Table 1: Protocol steps, what each relies on, and what we implemented.

Step	Mechanism	Relies on	Status
Provider commits to activations	TOPLOC polynomial commitment, last layer	cryptographic binding of the proof to its hash	implemented
	per-layer commitment (28 proofs)	same	measured separately, not integrated
Commitment bound to the served text	validator recomputes on the served tokens	validator honesty	implemented
Commitment bound to the model that generated the text	none	—	not provided (Section 7)
Verdict	last-layer thresholds; per-layer classifier	statistical separation, validator honesty	implemented
Authorised kernel	declaration or enforcement	not specified	proposed
Penalty	stake, bond, dispute window	smart contract; single trusted validator	implemented (local chain)

5 Experimental setup

Table 2 lists the setup and Table 4 the controls. Each prompt gets a greedy 256-token answer with the end-of-sequence token suppressed, so every sequence has eight checkpoints; in a 20-prompt probe without suppression, 19 answers ran to 256 tokens anyway. The two trained modifications see 2,400 Alpaca [14] examples, with effective batch 8 and sequence length 512.

Table 2: Setup of the main experiments.

Model	Qwen2.5-1.5B-Instruct [11], rev. `989aa79`
Shape	28 layers, hidden size 1536, bf16
Hardware	NVIDIA A10G (24 GB)
Software	PyTorch 2.11, CUDA 12.8, transformers 5.17, `toploc` 0.1.6, TF32 off
Prompts	200 from UltraChat-200k [5]
Answers	greedy, 256 tokens, 8 checkpoints
Reference	SDPA attention, batch 1
Honest	eager; batch 4; batch 4 + 17 padding
Tampered	4 models (Table 3)
Splits	5-fold CV grouped by prompt, or a train/test split by prompt id (stated per table)
Threshold	1% FRR on training prompts only

Table 3: Tampered models. All four write coherent text on a sanity prompt.

Class	Variant	Construction
1	8-bit	bitsandbytes LLM.int8() [3], outlier threshold 6.0
2	4-bit	bitsandbytes NF4 [4], bf16 compute
3	LoRA	r=16, α=32 on Q/K/V, lr 2×10^{-4}, 300 steps, merged [9]
4	Fine-tune	all weights, lr 10^{-5}, 8-bit AdamW, 300 steps

Table 4: Controls and the confound each addresses.

Control	Confound
Grouped by prompt	same prompt in train and test
Shared text	classifier learns the text, not the weights
Pre-EOS checkpoints	effect of suppressing end of sequence
Decode-time activations	prefill on both sides is unrealistic
Layer, feature, checkpoint, k ablations	where the signal comes from
Held-out modification	closed-set accuracy mistaken for integrity
Held-out honest kernel	classifier only knows seen kernels
Kernel attack	provider exploits authorised variation

6 Results

6.1 The last-layer check under honest kernel noise

Table 5 shows the check with three sets of thresholds. Calibrated on all three honest configurations, it keeps a 2% FPR on held-out prompts but flags no quantised sequence and only 55–60% of fine-tuned ones. The published thresholds reject 23% of honest prompts. Calibrating without eager traffic gives much tighter thresholds, which flag every tampered sequence but reject all 200 eager prompts and 4% of the remaining honest traffic. A second calibration run reproduced the first row exactly.

Table 5: TOPLOC's last-layer check under three sets of thresholds $(T_{\text{exp}}, T_{\text{mean}}, T_{\text{median}})$. FPR is on held-out prompts of the calibration configurations; the other columns cover all 200 prompts.

			Detection (%) ↑			
Calibrated on	FPR (%) ↓	Eager rejected (%) ↓	8-bit	4-bit	LoRA	Fine-tune
All three configs $(46.02, 25.80, 20.0)$	2	6	0	0	59.5	54.5
Batch-shape only $(7.01, 1.80, 1.0)$	4	100	100	100	100	100
Published $(38, 10, 8)$	23	69	0	50	100	100

Table 6 explains why. On all three statistics, eager attention moves the last layer further than 4-bit quantisation does, and 8-bit quantisation falls between batch-size noise and eager noise. These results do not contradict TOPLOC's. Our batch-size noise falls inside the range TOPLOC reports, and the published thresholds raise no false alarms on it. Eager attention is the outlier: its mantissa statistics are more than twice the largest TOPLOC reports. Our model is also smaller; with a hidden size of 1536, the 128 recorded entries cover 8.3% of the hidden state, against at most 3.6% for the models TOPLOC tested.

Table 6: Last-layer statistics for honest configurations and tampered models, as the mean over 200 prompts of each sequence's maximum. TOPLOC's range spans the maxima in its Table 2 [10].

	Source	Exponent mismatches	Mantissa mean	Mantissa median
Honest	Eager attention	25.55	14.22	10.64
	Batch 4	4.35	1.35	1.00
	Batch 4 + 17 padding	4.30	1.37	1.02
Tampered	8-bit	8.88	3.47	2.71
	4-bit	21.67	10.59	8.59
	LoRA	43.19	25.70	22.72
	Fine-tune	41.05	24.05	20.46
TOPLOC honest range		12–28	3.15–6.30	2–4
Published thresholds		38	10	8

6.2 Attribution with the per-layer profile

With the full profile, the classifier accepts both kernels and still separates the known modifications. Table 7 gives the out-of-fold result on matched text, with each honest configuration as a separate example. Balanced accuracy is 99.3%; the 95% t-interval across the five folds is 98.7–99.9%. This interval reflects variation between prompt folds within one model, GPU and software stack, not variation between deployments. Per-class precision and recall are at least 0.96 for every class (Table 8). Most errors fall on honest eager traffic: 13 of 200 prompts (6.5%) are labelled as quantised. Table 9 lists other settings; the pooled one we fixed before the experiment averages each prompt's three honest profiles and scores 99.5%, but averaging hides the eager errors.

6.3 Controls and ablations

A tampered model writes different text, and the classifier might learn the text. When both models read identical tokens (shared text) balanced accuracy is 99.1%, and using only checkpoints before the natural end of the answer gives 98.8% in both settings (Table 9).

Figure 5 retrains the classifier on parts of the profile. The last layer alone, the information TOPLOC uses, gives 78.0%, flagging 74.5% of eager traffic and missing 27% of 8-bit sequences. The first 14 layers give 98.6% and the last 14 only 91.6%. The classifier's weights agree (Figure 6): layers

Table 7: Out-of-fold confusion matrix of the per-layer classifier on matched text, prefill on both sides. Rows are the true source; columns are the predicted class.

	Clean	8-bit	4-bit	LoRA	Fine-tune
Honest: eager attention	187	8	5	0	0
Honest: batch 4	199	0	0	0	1
Honest: batch 4 + padding	199	0	0	0	1
8-bit	1	199	0	0	0
4-bit	0	0	200	0	0
LoRA	0	0	1	199	0
Fine-tune	0	0	0	0	200

Table 8: Per-class precision and recall, same predictions as Table 7.

Class	Precision ↑	Recall ↑	Support
Clean	0.998	0.975	600
8-bit	0.961	0.995	200
4-bit	0.971	1.000	200
LoRA	1.000	0.995	200
Fine-tune	0.990	1.000	200

Table 9: Five-class balanced accuracy (%) by setting, with the 95% interval across the five folds. Separate: one honest example per configuration (main setting). Pooled: the three honest configurations averaged per prompt, as fixed before the experiment.

Text	Checkpoints	LR, separate ↑	LR, pooled ↑	k-NN, pooled ↑
Matched	all eight	99.3 (98.7–99.9)	99.5 (99.1–99.9)	98.8
Matched	pre-EOS	—	98.8 (97.4–100)	96.7
Shared	all eight	99.1 (98.5–99.6)	99.8 (99.5–100)	98.3
Shared	pre-EOS	—	98.8 (97.4–100)	97.3

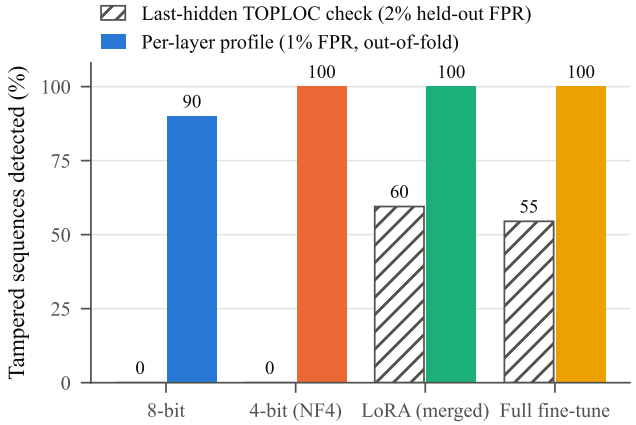


Figure 3: Detection rate. Hatched: last-layer check with thresholds calibrated on all honest configurations. Solid: per-layer classifier at 1% FPR, out of fold.

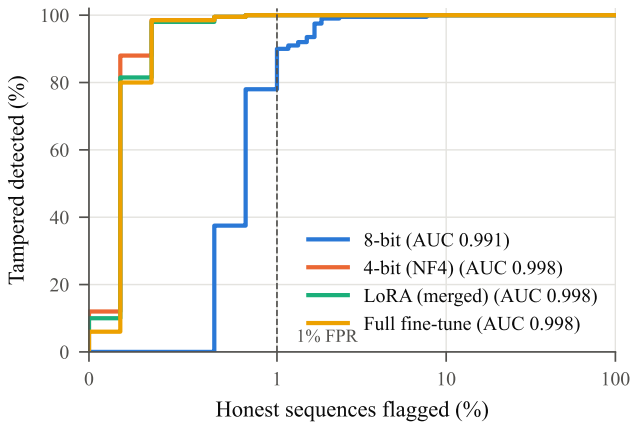


Figure 4: ROC of the score $1-P(\text{clean})$ for all honest sequences against each known modification.

0–4 carry 28% of the absolute standardised coefficient mass and layer 27 another 7%, against 3.6% for a uniform split. Index overlap and mantissa median carry most of it (31% each). Smaller k costs accuracy: 96.2% at k=16, 98.4% at 32, 98.6% at 64 and 99.3% at 128, mainly through more eager false alarms (12.5% at k=16 against 6.5% at 128).

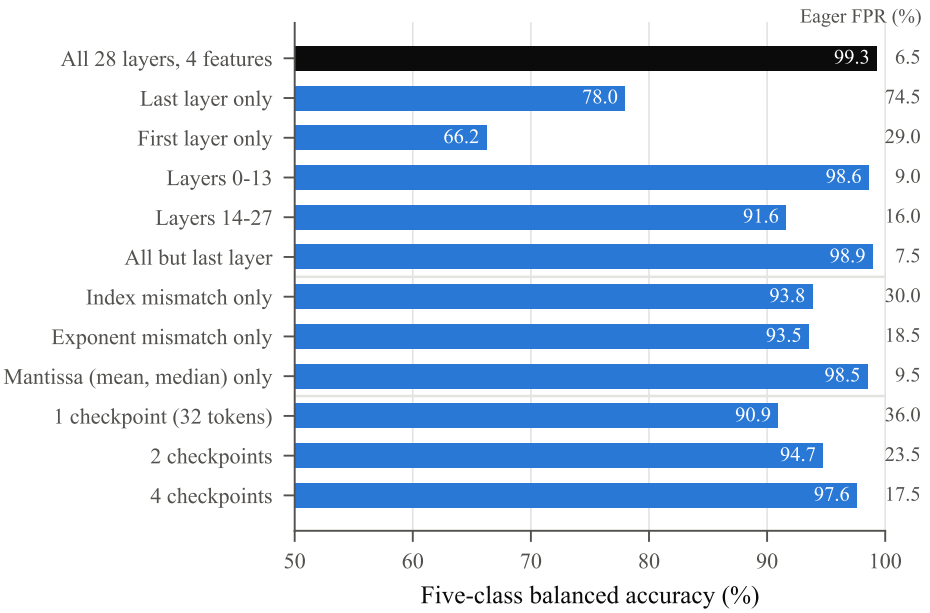


Figure 5: Ablations on matched text. Bars show five-class balanced accuracy; the right column shows the share of eager-attention prompts flagged.

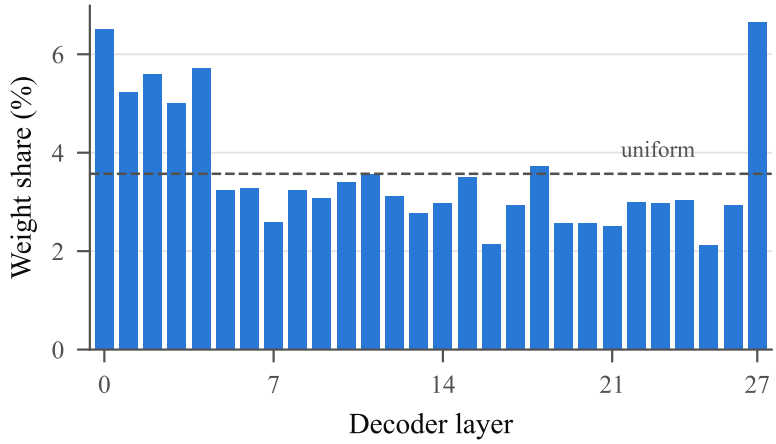


Figure 6: Share of the classifier's absolute standardised coefficients per layer, summed over classes and features.

6.4 Decode-time activations

A real provider would commit to activations recorded while decoding. We repeated the experiment with the provider side taken from decoding: each tampered model's own generations and teacher-forced decoding for the honest configurations. We added a fourth honest configuration, the reference model decoding on the reference kernel, so that the honest class includes the decode–prefill gap. The validator side stays a prefill with θ.

Cross-validated balanced accuracy is 99.0% (98.4–99.6%), against 80.6% for the last layer alone (Table 10). None of the reference model's own decodes is flagged; eager traffic is flagged on 12% of prompts. A classifier trained only on prefill profiles also works on decode-time profiles without retraining, at 99.2%.

Table 10: Per-layer classifier with the provider's activations recorded during decoding, 200 prompts per source. Out of fold except in the last row.

	Balanced	Honest flagged (%) ↓		Detected at 1% FPR (%) ↑	
Training data	accuracy (%) ↑	Ref. decode	Eager	8-bit	4-bit / LoRA / FT
Decode, all layers	99.0 (98.4–99.6)	0	12	85	100
Decode, last layer only	80.6 (79.7–81.5)	0	90	—	—
Prefill only (no retraining)	99.2	0	3	—	—

6.5 Operating point

Balanced accuracy averages over classes and decision rules; a verifier needs FAR and FRR at one threshold. Table 11 trains on prompts 0–99, chooses τ on those prompts only, and tests on prompts 100–199. Intervals are Wilson 95% intervals with the sequence as the sampling unit. At the 1% FRR target the verifier rejects 0.7% of honest prefill sequences but accepts 47% of 8-bit sequences; 4-bit, LoRA and fine-tune are never accepted. On decode-time data the 8-bit FAR falls to 22% at an FRR of 1.8%. Taking the most likely class instead of thresholding $P(\text{clean})$ accepts no 8-bit prefill sequence but rejects 4.3% of honest ones. 8-bit quantisation is the binding case: its profile is the one closest to honest eager traffic, so any operating point trades 8-bit acceptance against eager rejection. $P(\text{clean})$ is reasonably calibrated on the test split (expected calibration error 0.021, Brier score 0.015).

Table 11: Verdicts at a fixed threshold. Train prompts 0–99, test prompts 100–199 (100 per source); τ set for 1% FRR on training prompts. FRR: honest sequences rejected. FAR: tampered sequences accepted. Brackets: Wilson 95% intervals.

	FRR (%) ↓	FAR (%) ↓			
Provider activations	all honest	8-bit	4-bit	LoRA	Fine-tune
Prefill, threshold	0.7 [0.2–2.4]	47 [37.5–56.7]	0 [0–3.7]	0 [0–3.7]	0 [0–3.7]
Prefill, most likely class	4.3 [2.5–7.3]	0 [0–3.7]	0 [0–3.7]	0 [0–3.7]	0 [0–3.7]
Decode, threshold	1.8 [0.9–3.6]	22 [15.0–31.1]	0 [0–3.7]	0 [0–3.7]	0 [0–3.7]
Decode, most likely class	3.2 [1.9–5.5]	3 [1.0–8.5]	0 [0–3.7]	0 [0–3.7]	0 [0–3.7]

6.6 Unknown modifications

A verifier must reject modifications it was never trained on. We left each known modification out of training in turn and tested whether it is accepted as clean (Table 13). Held-out 4-bit, LoRA and fine-tuned sequences are still rejected, with at most 1 of 100 accepted; the classifier simply gives them a neighbouring label. Held-out 8-bit quantisation is accepted on all 100 test prompts, with prefill or decode-time data. The classifier separates 8-bit from clean only because it has seen 8-bit; it does not learn a general notion of "not θ".

We also served six modifications that never appeared in training (Table 12): the LoRA model loaded with 4-bit weights, and 4-bit quantisation of only some layers (the first 14, the last 14, even layers, odd layers, or only the MLPs). These share 4-bit quantisation with a training class, so they test variation around a known modification rather than an unrelated one. With the detector restricted to SDPA traffic, none of the 300 test sequences was accepted. The detector that also accepts eager traffic accepted 4 of 50 sequences quantised in the first 14 layers. The mixed-kernel last-layer check caught 66% of the LoRA model but at most 2% of the partial quantisations.

Table 12: Modifications never seen in training, test prompts 150–199 (50 sequences each), decode-time activations, threshold rule. Detectors trained on prompts 0–149. Brackets: Wilson 95% intervals.

	Share of linear	Per-layer FAR (%) ↓		Last-layer detected (%) ↑		
Modification	params in 4-bit	Unrestricted	SDPA only	All configs	Batch-shape only	Published
LoRA + 4-bit	100%	0 [0–7.1]	0 [0–7.1]	66	100	100
4-bit, layers 0–13	50%	8 [3.2–18.8]	0 [0–7.1]	0	100	26
4-bit, layers 14–27	50%	0 [0–7.1]	0 [0–7.1]	0	100	16
4-bit, even layers	50%	0 [0–7.1]	0 [0–7.1]	0	100	10
4-bit, odd layers	50%	0 [0–7.1]	0 [0–7.1]	2	100	12
4-bit, MLPs only	88%	0 [0–7.1]	0 [0–7.1]	2	100	40

Table 13: Held-out modification accepted as clean (FAR, %), threshold rule, test prompts 100–199, 100 sequences each, with Wilson 95% intervals. FRR on honest test traffic was 0.5–1.8%.

Held out	Prefill	Decode
8-bit	100 [96.3–100]	100 [96.3–100]
4-bit	1 [0.2–5.4]	1 [0.2–5.4]
LoRA	0 [0–3.7]	0 [0–3.7]
Fine-tune	0 [0–3.7]	0 [0–3.7]

6.7 Configuration-adaptive provider (A2)

Because eager traffic counts as honest, a provider can tamper with the weights and serve the result with eager attention. We replayed each tampered model's own generations by decoding, with eager attention at batch 1 and with SDPA at batch 4, on prompts 100–199.

Table 14 uses a classifier trained on decode-time data from prompts 0–99 without attack examples, taking the most likely class. Eager attention hides the quantisation: 77% of 8-bit and 66% of 4-bit sequences are called clean, and LoRA and fine-tuned sequences, though still flagged on 71–78% of prompts, usually get the wrong label. The last-layer check with mixed-kernel thresholds catches only 12% of eager-served 8-bit. Batch 4 does not help the attacker; 96–100% of those sequences are caught and correctly named.

Table 14: Tampered models served with a non-reference kernel, 100 held-out prompts each. Per-layer: most likely class from a classifier trained on decode-time data from prompts 0–99 without attack examples. Last-layer: the thresholds of Table 5.

		Per-layer (%)		Last-layer detected (%) ↑		
Kernel	Variant	Detected ↑	Called clean ↓	All configs	Batch-shape only	Published
Eager, batch 1	8-bit	23	77	12	100	77
	4-bit	34	66	64	100	99
	LoRA	71	29	91	100	100
	Fine-tune	78	22	77	100	100
SDPA, batch 4	8-bit	96	4	0	100	1
	4-bit	100	0	0	100	50
	LoRA	100	0	59	100	100
	Fine-tune	100	0	57	100	100

Kernel policy. Table 15 compares three policies at the threshold rule, training on prompts 0–149 and testing on 150–199 (50 sequences per cell). Under the *unrestricted* policy, one detector accepts every authorised kernel; it accepts 92% of eager-served 8-bit and 68% of eager-served 4-bit. Under a

declared policy, each provider names its kernel and the verifier uses a detector trained for that kernel, including on eager-served tampering; the attacker declares whichever kernel is accepted more often. This catches eager-served 4-bit, LoRA and fine-tuning (accepted on 2–8% of prompts) while rejecting 2% of honest eager traffic, but it still accepts 96% of eager-served 8-bit. A declaration is also only a claim: nothing in our protocol checks it. Under an *enforced* policy, the provider must serve with SDPA; the verifier then accepts none of the 400 tampered test sequences, at the default batch or at batch 4, and rejects none of the 150 honest ones. With 50 sequences per cell, the 95% upper bound on each of these zero rates is 7.1%. In this experiment the validator's reference is always an SDPA prefill, since we did not record eager reference prefills. Adding attack examples to a single detector for all kernels instead raises eager-attack detection to 62% for 8-bit and 98–100% for the others, but it then flags 38% of honest eager traffic.

Table 15: Kernel policies, threshold rule, test prompts 150–199 (50 sequences per cell). FAR on eager-served tampering; under the declared policy the attacker's better declaration is shown. Brackets: Wilson 95% intervals.

	FRR (%) ↓		FAR on eager-served tampering (%) ↓			
Policy	eager	SDPA configs	8-bit	4-bit	LoRA	Fine-tune
Unrestricted	4 [1.1–13.5]	0 [0–7.1]	92 [81.2–96.8]	68 [54.2–79.2]	28 [17.5–41.7]	28 [17.5–41.7]
Declared	2 [0.4–10.5]	0 [0–7.1]	96 [86.5–98.9]	6 [2.1–16.2]	2 [0.4–10.5]	8 [3.2–18.8]
Enforced SDPA	not allowed	0 [0–7.1]	not possible; SDPA-served tampering: 0 [0–7.1] for all 8 attacks			

Unseen honest kernels. The classifier only accepts kernels it has seen. Left out of training, a batch-size configuration is still accepted (0.5% flagged), but all 200 eager prompts are flagged, mostly as 4-bit. A small honest-only check on an NVIDIA L4 raised no false alarms on 50 sequences under either check; we did not run tampered models on the L4.

6.8 Detector-guided quantisation (A3)

An attacker who can query the detector can choose what to quantise. We gave the attacker the five partial quantisations of Table 12, each covering at least half of the linear parameters, and let it score them on calibration prompts 0–19 with the detector it faces, picking the one accepted most often (ties broken by mean $P(\text{clean})$). Against the SDPA-only detector no candidate was accepted on any calibration prompt; the search picked the last-14-layer variant, which was accepted on 0 of 50 test prompts. Against the detector that also accepts eager traffic, the search picked the first-14-layer variant (accepted on 1 of 20 calibration prompts), which was accepted on 4 of 50 test prompts (8%, [3.2–18.8]). This is a weak attacker with five discrete choices, and it does not show robustness to an attacker that optimises the weights against the detector.

6.9 A second model

We repeated the core experiment on SmolLM2-1.7B-Instruct, a Llama-architecture model from a different family (24 layers, hidden size 2048, revision `31b70e2`), with 100 prompts, 128-token answers and decode-time activations. Honest traffic was the reference decode, eager attention and batch 4; tampered traffic was 4-bit and 8-bit quantisation, and the eager attack used prompts 50–99.

The picture differs from Qwen (Table 16). Eager attention barely changes this model: its mean index mismatch is within 0.002 of the reference decode at every layer, while 8-bit and 4-bit sit at 0.08–0.12 and 0.14–0.22 respectively across layers. The full profile reaches 98.7% balanced accuracy and the last layer alone 98.4%; no honest sequence is flagged and no tampered sequence is called clean in cross-validation. At the threshold rule, trained on prompts 0–49 and tested on 50–99, eager-served

quantisation is never accepted, and a detector trained without eager traffic still accepts all honest eager traffic. On this model, then, there is no kernel loophole and no need for the full profile. Whether a model shows the overlap seen on Qwen has to be measured per model.

Table 16: SmolLM2-1.7B-Instruct, decode-time activations. CV: 5-fold grouped by prompt, 100 prompts. FAR and FRR: train prompts 0–49, test 50–99 (50 per cell), threshold rule, detector accepting all honest configurations.

Balanced accuracy, all 24 layers (CV)	98.7%
Balanced accuracy, last layer only (CV)	98.4%
FRR: reference / eager / batch 4	0 / 0 / 2%
FAR: 4-bit / 8-bit	0 / 0%
FAR: eager-served 4-bit / 8-bit	0 / 0%

6.10 Cost

Commitment. We built real TOPLOC commitments for all 28 layers of 10 reference generations (256 tokens each, k=128). One commitment per layer takes 2,064 bytes per sequence, the same as TOPLOC's last-layer commitment, so the full profile costs 57,792 bytes per 256 tokens, or 226 bytes per token. Building all 28 commitments took 0.22 s and verifying them 0.29 s per sequence on the instance's CPU, and capturing every layer during the validator's prefill added no measurable time to its 0.042 s. We did not integrate these commitments into the contract or measure them under load.

Serving. Table 17 shows decoding throughput at three batch sizes. At batch 1, 8-bit is 4.9× and 4-bit 1.3× slower than bf16, as in our earlier measurement, so the time saving Δ is negative. At batch 32, 4-bit reaches 96% of bf16 throughput with 29% less peak memory, which leaves room for more replicas or larger batches per GPU; 8-bit stays three times slower. Whether cheating pays therefore depends on the serving load, and the condition $\Delta < pS$ has to be evaluated at the provider's own operating point.

Table 17: Decoding throughput (tokens/s, all sequences) and peak memory on an A10G, 128 new tokens, mean of two runs after one warm-up.

Batch	bf16	8-bit	4-bit
1	39.1	8.0	29.4
8	258.3	55.3	203.1
32	494.2	163.9	476.5
Peak MiB, batch 32	6209	6236	4413

We also ran the whole pipeline once on a local Anvil chain: a provider staked 100 mock tokens and served the fine-tuned model, the last-layer check flagged the sequence (57 exponent mismatches against 46.02), the classifier returned class 4, and the contract zeroed the stake and recorded `[4]`.

7 Security analysis

What the 99.3% measures. It is closed-set attribution among four modifications, in one model, on one GPU type, with prompt folds as the only source of variation. It does not bound the acceptance of unknown modifications, which can be complete (Section 6.6), and the fold interval is not a deployment interval.

Commitments and selective disclosure. A TOPLOC commitment binds activations to the tokens the validator recomputes, so a provider cannot report activations that do not match those tokens under θ. It does not bind the tokens to the weights that chose them. A provider can generate with $\tilde{\theta}$, then run one prefill with θ on its own output and commit to those activations; both checks then pass. This costs one prefill per sampled response, far less than generation, and it is the same gap TOPLOC reports for speculative decoding. Closing it needs a binding between sampling and the committed weights, for example output-distribution tests on the served text [6] or trusted execution [1]. A provider that serves verification requests differently from ordinary ones is also outside what any per-response check can see, unless verification requests are indistinguishable from traffic.

Kernel enforcement. Enforcing the kernel closed the A2 attack in our data, but we have not shown how to enforce it. A declared kernel is a claim; verifying it needs either a kernel signature in the activations, which our eager results suggest exists but which an attacker could imitate, or attested execution. Other execution choices we did not vary, such as precision, continuous batching and tensor parallelism, could open similar gaps.

Detector-aware attacks. Our A3 attacker searches over a small, discrete set of layer subsets. An attacker that optimises the weights directly against a known linear detector, for example by fine-tuning with a penalty on the detector's score, is likely to do better, and we did not test it.

Generalisation and statistics. Our evidence covers two models of 1.5–1.7B parameters, one GPU type for tampering, and 50–200 prompts per source. The second model behaved differently from the first, so neither the kernel loophole nor the value of the full profile can be assumed for a new model. Wilson intervals on 50–100 sequences cannot certify rates below a few percent; a zero count out of 50 only bounds the rate below 7.1%.

8 Conclusion

Honest kernel noise and weight tampering can look the same at the last layer of a language model and still differ across its depth. On Qwen2.5-1.5B-Instruct, a per-layer profile of TOPLOC's statistics attributes four known modifications with 99.3% balanced accuracy while accepting two attention kernels, where the last-layer check with mixed-kernel thresholds catches no quantisation. The profile is not an integrity guarantee. It accepts an unseen 8-bit model on every test prompt, a provider that switches to eager attention passes on most quantised prompts, and neither check binds the served text to the weights that generated it. With the kernel enforced, no tested attack passed, including six unseen modifications and a detector-guided choice of layers, which suggests that verification protocols should fix the execution configuration rather than tolerate it. On SmolLM2-1.7B the overlap between kernel noise and quantisation did not occur, so the need for a per-layer profile is a property of the model, to be measured before deployment. A per-layer commitment is cheap to build and check, but it is 28 times larger than TOPLOC's. Table 18 summarises what is established.

References

[1] Will Cai, Tianneng Shi, Xuandong Zhao, and Dawn Song. Are you getting what you pay for? auditing model substitution in LLM APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.04715.

Table 18: Status of the paper's claims.

Claim	Status
Last-layer check misses quantisation under mixed-kernel thresholds	measured
Per-layer attribution of 4 known modifications	measured
Holds for decode-time activations	measured
Rejects unknown modifications	not established (fails for 8-bit)
Robust to kernel choice (A2)	not established
Kernel enforcement closes A2	measured if enforced; enforcement proposed
Robust to detector-aware attacks (A3)	one discrete search: no evasion with enforced kernel; weight optimisation untested
Other models and hardware	second model: attribution holds, loophole absent; tampering on other GPUs untested
Per-layer commitment is practical	commitment measured; not integrated
Penalty enforcement	implemented, trusted validator

[2] KD Conway, Cathie So, Xiaohang Yu, and Kartin Wong. opML: Optimistic machine learning on blockchain. *arXiv preprint arXiv:2401.17555*, 2024.

[3] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[6] Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.20247.

[7] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. SafetyNets: Verifiable execution of deep neural networks on an untrusted cloud. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, 2025.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[10] Jack Min Ong et al. TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. arXiv:2501.16007.

[11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[12] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024. arXiv:2404.16109.

[13] Yifan Sun, Yuhang Li, Yue Zhang, Yuchen Jin, and Huan Zhang. SVIP: Towards verifiable inference of open-source large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2410.22307.

[14] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

[15] Jason Teutsch and Christian Reitwießner. A scalable verification solution for blockchains. *arXiv preprint arXiv:1908.04756*, 2019.

[16] Florian Tramèr and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In *International Conference on Learning Representations (ICLR)*, 2019.

A Reproducibility

- Code and data: https://github.com/bruce249/proof-of-weights.
- Model revision: `989aa7980e4cf806f80c7fef2b1adb7bc71aa306`.
- `toploc` 0.1.6, PyTorch 2.11.0, transformers 5.17.0.
- Seeds: prompts and classifier 20260923, LoRA 202609231, fine-tune 202609232.
- Analysis scripts in `paper/`: `analysis.py`, `decode_analysis.py`, `review_analysis.py`, `review_gpu_analysis.py`.
- GPU runs in `inference/`: `task02.py`, `task04.py`, `recalibrate.py`, `kernel_attack.py`, `review_gpu.py`.
- Proofs are built from CPU copies of the activations, because the library failed on CUDA tensors in our environment.
- The thresholds, classifier settings and pooled clean class were fixed before the experiments. All other analyses, including every result in Sections 6.5 to 6.7, were added afterwards and are exploratory.